1. Linear search: read, trace, explain

TEACHING KEY - public lesson resource, not a secure examination

WAGBA: Follow and explain search algorithms so we can choose the correct steps to find an item.

The library wants book number 13. Follow this early-stopping algorithm, not just where you can see 13 in the list.

items = the list below. target = the requested number. Indexes stay unchanged.

```
Index:   0   1   2   3   4   5   6
Value:   5   9  13  18  24  31  42
```

```
01  position ← 0
02  found ← False
03  WHILE position < LEN(items) AND found = False
04      IF items[position] = target THEN
05          found ← True
06      ELSE
07          position ← position + 1
08      ENDIF
09  ENDWHILE
10  OUTPUT found
```

Read: underline the comparison, circle the instruction that advances the index, and mark the stopping condition.

Index	Value	Match?	Next action
0	5	No	Check index 1
1	9	No	Check index 2
2	13	Yes	Set found True; stop

Explain: At index __, the current value is __. This equals target __, so the algorithm __.
At index 2, the value is 13. It matches target 13, so found becomes True. The WHILE condition is then false and this search stops.

Correct this claim: "The first value is not the target, so output not found."
Only one item was checked. Continue until a match is found or all items have been exhausted. For target 20, all seven values are checked and found remains False.

2. Binary search: justify the boundary

TEACHING KEY - public lesson resource, not a secure examination

WAGBA: Follow and explain search algorithms so we can choose the correct steps to find an item.

The list is sorted in ascending order. Search for 13. Keep original indexes. mid is an index; items[mid] is a value. items = the list below. target = the requested number. Indexes stay unchanged.

```
Index:   0   1   2   3   4   5   6
Value:   5   9  13  18  24  31  42
```

```
01  low ← 0
02  high ← LEN(items) - 1
03  found ← False
04  WHILE low ≤ high AND found = False
05      mid ← (low + high) DIV 2
06      IF items[mid] = target THEN
07          found ← True
08      ELSE
09          IF items[mid] < target THEN
10              low ← mid + 1
11          ELSE
12              high ← mid - 1
13          ENDIF
14      ENDIF
15  ENDWHILE
16  OUTPUT found
```

low	high	mid	Value	Next decision
0	6	3	18	Keep 0-2; high = 2
0	2	1	9	Keep 2-2; low = 2
2	2	2	13	found True; stop

Explain the first change: Target __ is smaller than value __. Because the list is sorted, keep indexes __ to __. Therefore high becomes __.

13 < 18. All values at indexes 3-6 are too large. Keep 0-2; high = mid - 1 = 2. Do not compare target 13 with midpoint index 3.

Why exclude a checked midpoint when it does not match? It is known not to match. Using mid + 1 or mid - 1 removes it and makes progress.

3. Independent comparison and reflection

TEACHING KEY - public lesson resource, not a secure examination

WAGBA: Follow and explain search algorithms so we can choose the correct steps to find an item.

Use the same list. Search for target 42 with both supplied algorithms. Do not reopen the completed model until you have tried.

items = the list below. target = the requested number. Indexes stay unchanged.

```
Index:   0   1   2   3   4   5   6
Value:   5   9  13  18  24  31  42
```

Linear: write the checked values, the result index and the number of item checks.

5, 9, 13, 18, 24, 31, 42. Found at index 6 after 7 item checks.

Binary: write each low/high/mid value and checked item, then state the result.

low/high/mid: 0/6/3 checks 18; 4/6/5 checks 31; 6/6/6 checks 42. Found at index 6 after 3 item checks.

Explain the difference: Both algorithms __. Linear search __, whereas binary search __. Binary search needs sorted data because __.

Both look for a target and can report found or not found. Linear checks sequentially and also works on unsorted data. Binary uses sorted order to rule out sections. It does not delete or renumber the list.

Challenge: Does this prove binary always uses fewer checks? Find a counterexample.

Target 5: linear checks one item; binary checks 18, 9, 5 (three items). Target 9 gives two checks for both. Count item inspections, not every operator.

My evidence and next action: I can __ independently / with __ support. Next I will __.

Look for accurate first traces, justified boundary changes and independent transfer. Model use and corrections are valuable, but not the same as first-attempt independence.